

# Análisis de Algoritmos 2026-1

## Tarea 3

June 23, 2026

| Expediente | Nombre                          |
|------------|---------------------------------|
| 219208106  | Bórquez Guerrero Angel Fernando |

---

## 1 Evaluación de polinomios de grado n

### 1.1 Algoritmo 1 - Potencia con ciclo anidado

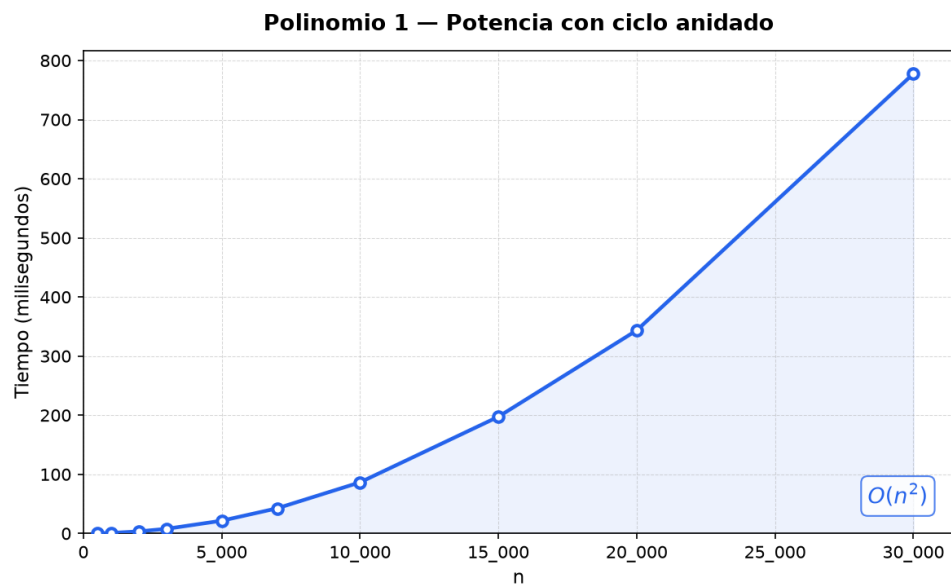
#### 1.1.1 Código

```
1 double Polinomio1(const vector<double>& C, double X)
2 {
3     int N = C.size() - 1;
4
5     double S = C[0];
6
7     for(int i = 0; i <= N; ++i)
8     {
9         double XN = 1.0;
10
11         for(int j = 1; j <= i; ++j)
12         {
13             XN *= X;
14         }
15
16         S += C[i] * XN;
17     }
18
19     return S;
20 }
```

### 1.1.2 Tiempos de ejecución

| $n$    | Tiempo ( $O(n^2)$ ) |
|--------|---------------------|
| 500    | 201.0740 $\mu$ s    |
| 1,000  | 844.7280 $\mu$ s    |
| 2,000  | 3.4778 ms           |
| 3,000  | 7.7801 ms           |
| 5,000  | 21.5232 ms          |
| 7,000  | 42.4610 ms          |
| 10,000 | 86.4387 ms          |
| 15,000 | 197.4038 ms         |
| 20,000 | 343.5000 ms         |
| 30,000 | 777.6184 ms         |

### 1.1.3 Gráfica de tiempos



## 1.2 Algoritmo 2 - Potencia recursiva

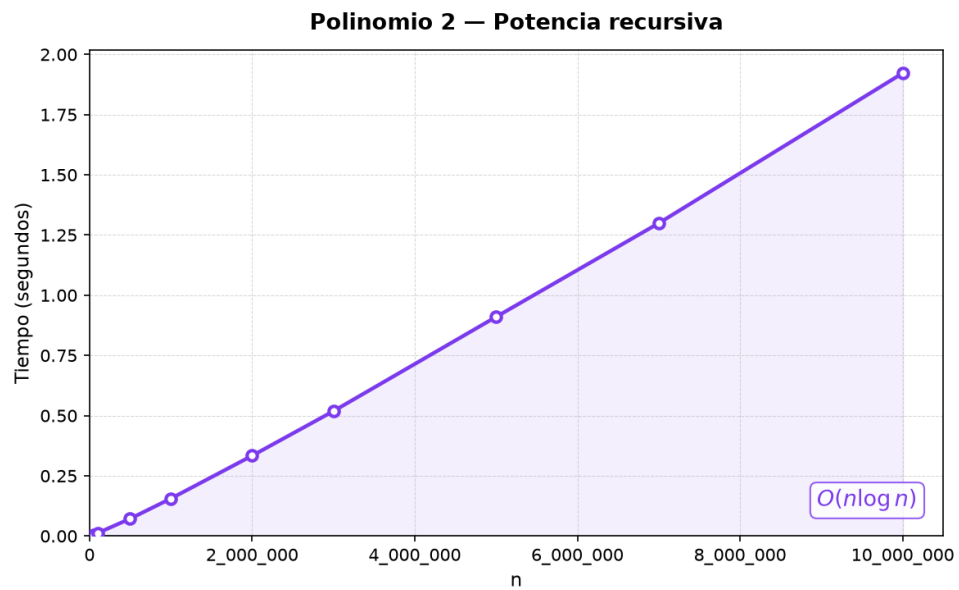
### 1.2.1 Código

```
1 double Potencia(double X, int j)
2 {
3     if(j == 0)
4     {
5         return 1.0;
6     }
7     else if(j % 2 == 1)
8     {
9         return X * Potencia(X, j - 1);
10    }
11    else
12    {
13        double t = Potencia(X, j / 2);
14
15        return t * t;
16    }
17 }
18
19 double Polinomio2(const vector<double>& C, double X)
20 {
21     int N = C.size() - 1;
22
23     double S = C[0];
24
25     for(int i = 0; i <= N; ++i)
26     {
27         S += C[i] * Potencia(X, i);
28     }
29
30     return S;
31 }
```

### 1.2.2 Tiempos de ejecución

| $n$        | Tiempo ( $O(n \log n)$ ) |
|------------|--------------------------|
| 10,000     | 857.8920 $\mu$ s         |
| 50,000     | 5.4186 ms                |
| 100,000    | 12.2009 ms               |
| 500,000    | 71.8926 ms               |
| 1,000,000  | 155.4319 ms              |
| 2,000,000  | 333.4747 ms              |
| 3,000,000  | 518.5661 ms              |
| 5,000,000  | 909.9825 ms              |
| 7,000,000  | 1.2990 s                 |
| 10,000,000 | 1.9221 s                 |

### 1.2.3 Gráfica de tiempos



## 1.3 Algoritmo 3 - Multiplicación acumulada

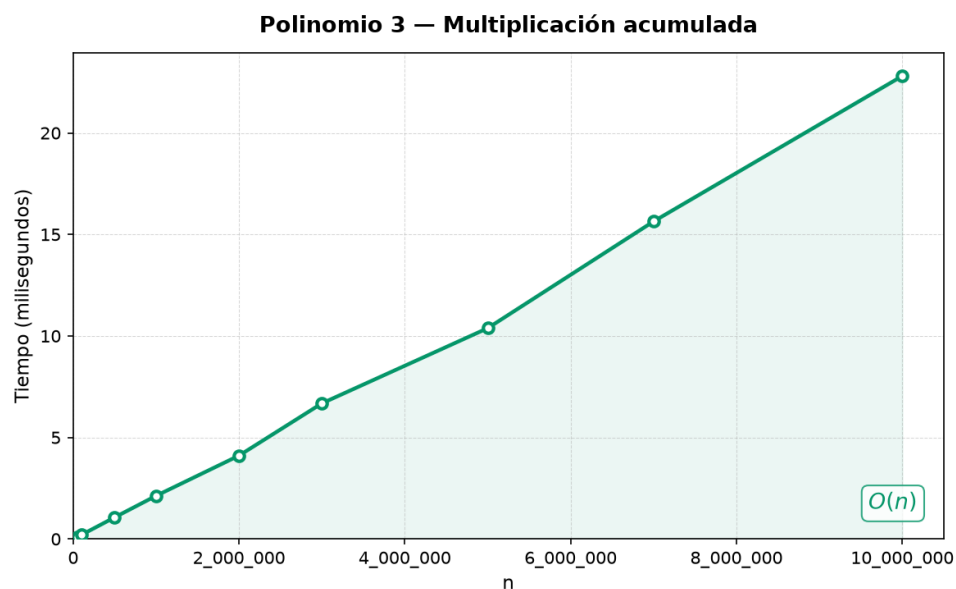
### 1.3.1 Código

```
1 double Polinomio3(const vector<double>& C, double X)
2 {
3     int N = C.size() - 1;
4
5     double XN = 1.0;
6     double S = C[0];
7
8     for(int i = 0; i <= N; ++i)
9     {
10         XN *= X;
11         S += C[i] * XN;
12     }
13
14     return S;
15 }
```

### 1.3.2 Tiempos de ejecución

| $n$        | Tiempo ( $O(n)$ ) |
|------------|-------------------|
| 10,000     | 20.4010 $\mu$ s   |
| 50,000     | 101.5430 $\mu$ s  |
| 100,000    | 202.8740 $\mu$ s  |
| 500,000    | 1.0646 ms         |
| 1,000,000  | 2.1265 ms         |
| 2,000,000  | 4.1067 ms         |
| 3,000,000  | 6.6811 ms         |
| 5,000,000  | 10.3838 ms        |
| 7,000,000  | 15.6404 ms        |
| 10,000,000 | 22.8040 ms        |

### 1.3.3 Gráfica de tiempos



## 1.4 Algoritmo 4 - Regla de Horner

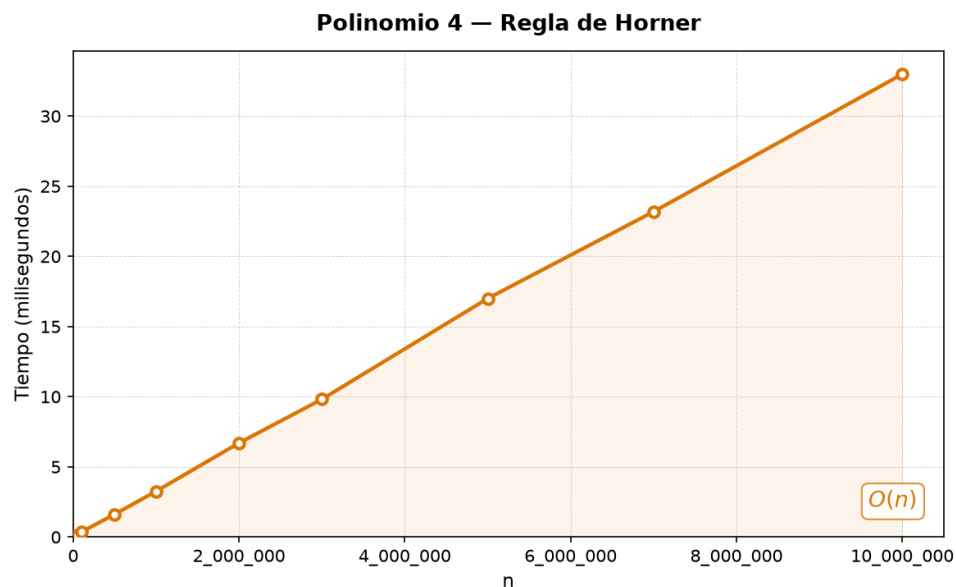
### 1.4.1 Código

```
1 double Polinomio4(const vector<double>& C, double X)
2 {
3     int N = C.size() - 1;
4     double S = 0.0;
5
6     for(int i = N; i >= 0; --i)
7     {
8         S = S * X + C[i];
9     }
10
11     return S;
12 }
```

### 1.4.2 Tiempos de ejecución

| $n$        | Tiempo ( $O(n)$ ) |
|------------|-------------------|
| 10,000     | 32.3010 $\mu$ s   |
| 50,000     | 161.2660 $\mu$ s  |
| 100,000    | 362.0410 $\mu$ s  |
| 500,000    | 1.6219 ms         |
| 1,000,000  | 3.2598 ms         |
| 2,000,000  | 6.7007 ms         |
| 3,000,000  | 9.8165 ms         |
| 5,000,000  | 16.9991 ms        |
| 7,000,000  | 23.1858 ms        |
| 10,000,000 | 32.9864 ms        |

### 1.4.3 Gráfica de tiempos



## 2 Sum

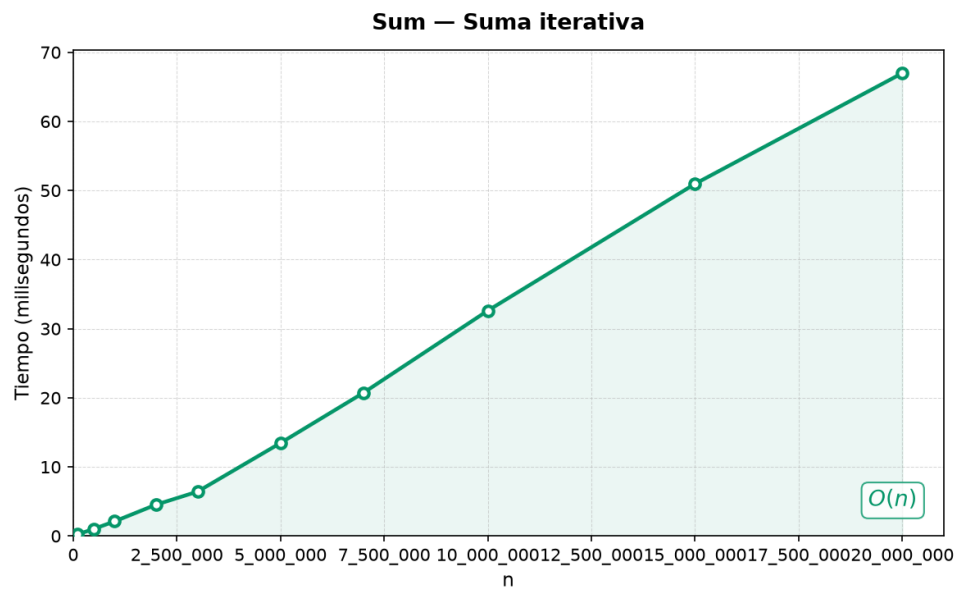
### 2.1 Código

```
1 float Sum(vector<float> a, int n)
2 {
3     float s = 0.0;
4
5     for(int i = 0; i < n; ++i) s += a[i];
6
7     return s;
8 }
```

## 2.2 Tiempos de ejecución

| $n$        | Tiempo ( $O(n)$ ) |
|------------|-------------------|
| 100,000    | 218.7250 $\mu$ s  |
| 500,000    | 1.0313 ms         |
| 1,000,000  | 2.1191 ms         |
| 2,000,000  | 4.5373 ms         |
| 3,000,000  | 6.4210 ms         |
| 5,000,000  | 13.4520 ms        |
| 7,000,000  | 20.7139 ms        |
| 10,000,000 | 32.6061 ms        |
| 15,000,000 | 50.9609 ms        |
| 20,000,000 | 66.9751 ms        |

## 2.3 Gráfica de tiempos



## 3 RSum

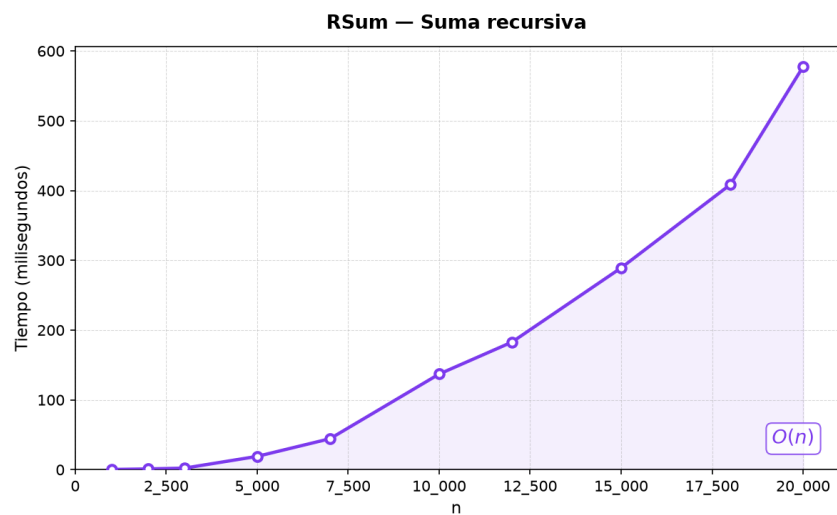
### 3.1 Código

```
1 float RSum(vector<float> a, int n)
2 {
3     float result;
4     if(n <= 0)
5     {
6         result = 0.0;
7     }
8     else
9     {
10        result = RSum(a, n - 1) + a[n-1];
11    }
12
13    return result;
14 }
```

### 3.2 Tiempos de ejecución

| $n$    | Tiempo ( $O(n)$ ) |
|--------|-------------------|
| 1,000  | 335.7120 $\mu$ s  |
| 2,000  | 1.0722 ms         |
| 3,000  | 2.1842 ms         |
| 5,000  | 19.0461 ms        |
| 7,000  | 44.3347 ms        |
| 10,000 | 137.0227 ms       |
| 12,000 | 182.7842 ms       |
| 15,000 | 289.1696 ms       |
| 18,000 | 408.5071 ms       |
| 20,000 | 577.4380 ms       |

### 3.3 Gráfica de tiempos





## 4 Add

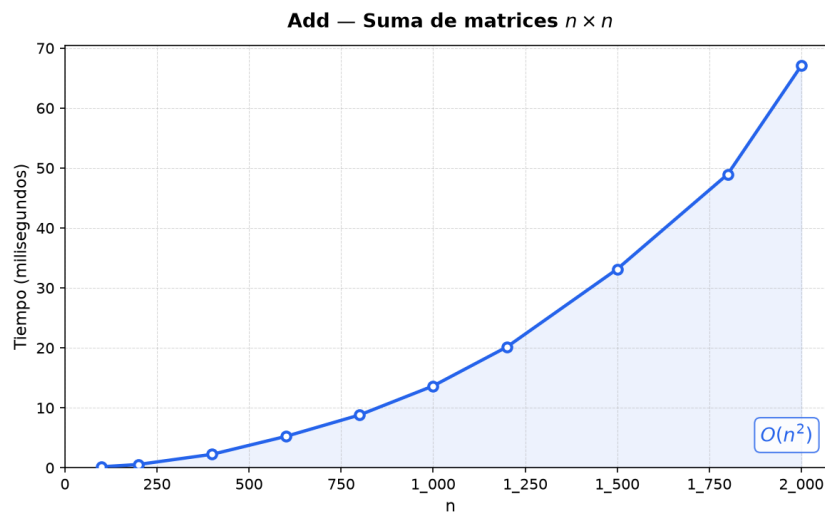
### 4.1 Código

```
1 vector<vector<float>> Add(vector<vector<float>> a, vector<vector<float>> b,  
2     int n)  
3 {  
4     vector<vector<float>> c(n, vector<float>(n));  
5  
6     for(int i = 0; i < n; ++i)  
7     {  
8         for(int j = 0; j < n; ++j)  
9         {  
10            c[i][j] = a[i][j] + b[i][j];  
11        }  
12    }  
13    return c;  
}
```

### 4.2 Tiempos de ejecución

| $n$   | Tiempo ( $O(n^2)$ ) |
|-------|---------------------|
| 100   | 165.8420 $\mu$ s    |
| 200   | 557.6420 $\mu$ s    |
| 400   | 2.2490 ms           |
| 600   | 5.2358 ms           |
| 800   | 8.8062 ms           |
| 1,000 | 13.6477 ms          |
| 1,200 | 20.1402 ms          |
| 1,500 | 33.1476 ms          |
| 1,800 | 48.9597 ms          |
| 2,000 | 67.0839 ms          |

### 4.3 Gráfica de tiempos



## 5 Trasp

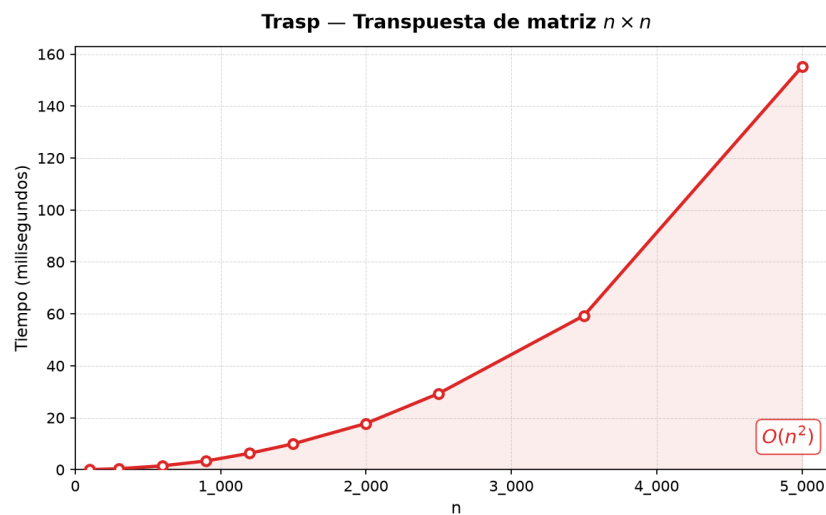
### 5.1 Código

```
1 void Trasp(vector<vector<float>> & a, int n)
2 {
3     float t = 0.0;
4
5     for(int i = 0; i < n-1; ++i)
6     {
7         for(int j = i+1; j < n; ++j)
8         {
9             t = a[i][j];
10            a[i][j] = a[j][i];
11            a[j][i] = t;
12        }
13    }
14 }
```

### 5.2 Tiempos de ejecución

| $n$   | Tiempo ( $O(n^2)$ ) |
|-------|---------------------|
| 100   | 41.5410 $\mu$ s     |
| 300   | 372.5950 $\mu$ s    |
| 600   | 1.4871 ms           |
| 900   | 3.3472 ms           |
| 1,200 | 6.3003 ms           |
| 1,500 | 9.9603 ms           |
| 2,000 | 17.7941 ms          |
| 2,500 | 29.3245 ms          |
| 3,500 | 59.3193 ms          |
| 5,000 | 155.1489 ms         |

### 5.3 Gráfica de tiempos



## 6 Mult

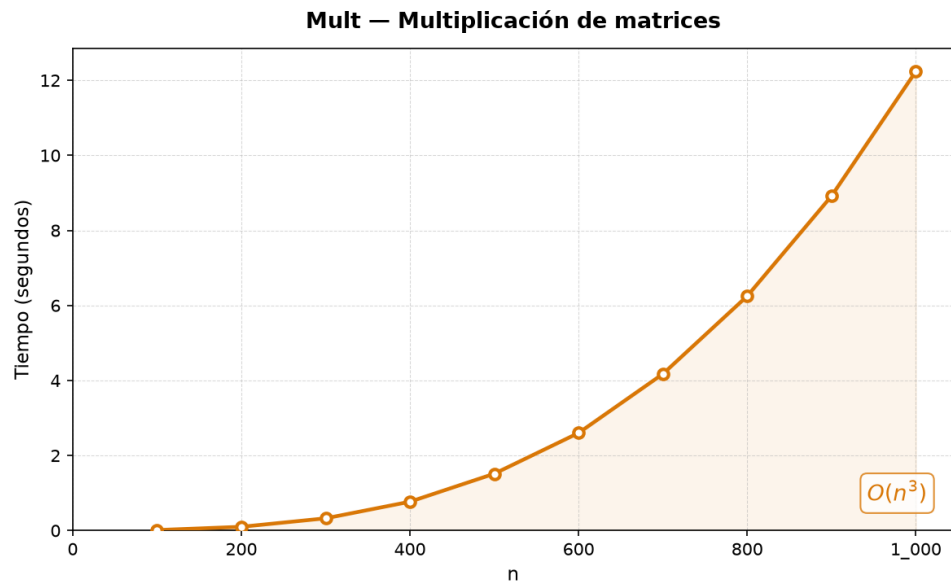
### 6.1 Código

```
1 vector<vector<float>> Mult(const vector<vector<float>>& a, const vector<
  vector<float>>& b)
2 {
3     int n = a.size();
4
5     vector<vector<float>> c(n, vector<float>(n, 0.0f));
6
7     for(int i = 0; i < n; ++i)
8     {
9         for(int j = 0; j < n; ++j)
10        {
11            for(int k = 0; k < n; ++k)
12            {
13                c[i][j] += a[i][k] * b[k][j];
14            }
15        }
16    }
17
18    return c;
19 }
```

### 6.2 Tiempos de ejecución

| $n$   | Tiempo ( $O(n^3)$ ) |
|-------|---------------------|
| 100   | 12.7125 ms          |
| 200   | 100.1668 ms         |
| 300   | 325.6764 ms         |
| 400   | 768.2177 ms         |
| 500   | 1.5144 s            |
| 600   | 2.6010 s            |
| 700   | 4.1743 s            |
| 800   | 6.2492 s            |
| 900   | 8.9168 s            |
| 1,000 | 12.2373 s           |

## 6.3 Gráfica de tiempos



## 7 Perm

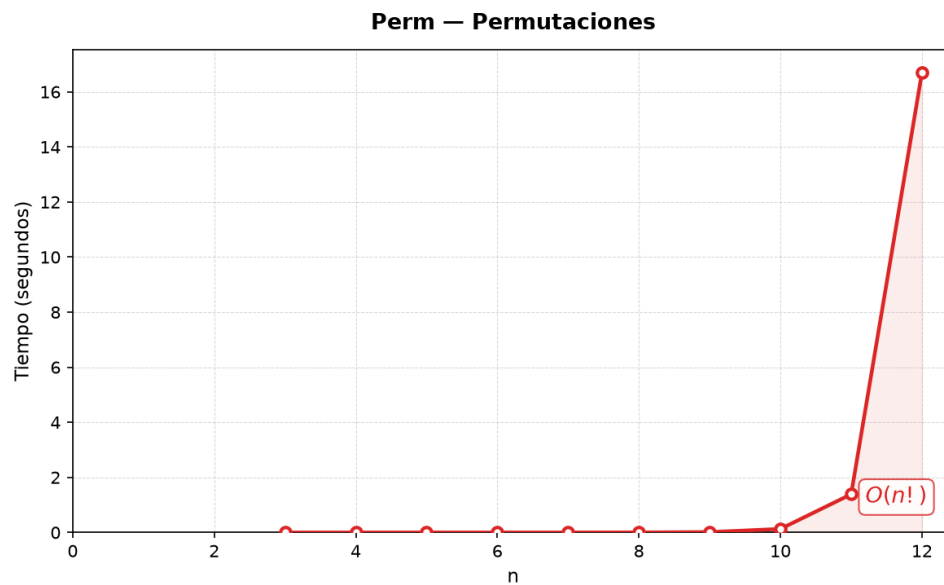
### 7.1 Código

```
1 void Perm(vector<int>& a, int k, int n)
2 {
3     if(k == n)
4     {
5         for(int i = 0; i < n; ++i)
6         {
7             cout << a[i] << " ";
8         }
9         cout << endl;
10    }
11    else
12    {
13        for(int i = k; i < n; ++i)
14        {
15            swap(a[k], a[i]);
16
17            Perm(a, k + 1, n);
18
19            swap(a[k], a[i]);
20        }
21    }
22 }
```

## 7.2 Tiempos de ejecución

| $n$ | Tiempo ( $O(n!)$ ) |
|-----|--------------------|
| 3   | 422 ns             |
| 4   | 1.1730 $\mu$ s     |
| 5   | 5.3820 $\mu$ s     |
| 6   | 25.0260 $\mu$ s    |
| 7   | 174.0150 $\mu$ s   |
| 8   | 1.4394 ms          |
| 9   | 13.4568 ms         |
| 10  | 127.1139 ms        |
| 11  | 1.3867 s           |
| 12  | 16.6943 s          |

## 7.3 Gráfica de tiempos



## 8 SeqSearch

### 8.1 Código

```
1 int SeqSearch(vector<int>& a, int x)
2 {
3     int n = a.size() - 1;
4     a[0] = x;
5     int i = n;
6
7     while(a[i] != x)
8     {
9         i = i - 1;
10    }
11    return i;
12 }
```

### 8.2 Tiempos de ejecución

| $n$         | Tiempo ( $O(n)$ ) |
|-------------|-------------------|
| 100,000     | 179.0700 $\mu$ s  |
| 500,000     | 895.7680 $\mu$ s  |
| 1,000,000   | 1.7590 ms         |
| 5,000,000   | 9.3681 ms         |
| 10,000,000  | 17.6167 ms        |
| 20,000,000  | 36.9621 ms        |
| 30,000,000  | 53.1168 ms        |
| 50,000,000  | 90.5206 ms        |
| 75,000,000  | 137.4871 ms       |
| 100,000,000 | 177.4072 ms       |

### 8.3 Gráfica de tiempos

